WAPH-Web Application Programming and Hacking

Instructor: Dr. Phu Phung

Student

Name: Jack Hanchin

Email: mailto:hanchijd@mail.uc.edu

Figure 1: Jack's Headshot

https://github.com/Hanchijd/Hanchijd.github.io

Individual Project 1

Front-end Web Development with a Professional Profile Website and API Integration on github.io cloud service

Overview and Requirements

In this project, you will expand front-end web development skills by developing a Professional Profile Website and deploying it on `github.io` cloud service. This project has general, non-technical, and technical requirements with grade distributions as follows.

Student's Overview and Outcomes

In this project, we develop a Professional Profile Website using a culmination of everything we've learned thus far in this class, focusing primarily on the front end. As seen below, this lab requires a great deal of general, technical and non-technical skills, including but not limited to GitHub, Javascript via JQuery and Ajax, Web API Integration, JavaScript cookies, use of an open-source CSS template/framework, dealing with VMs, and writing/formatting long reports.

###General requirements (30 pts): - Create and deploy a personal website on GitHub cloud (github.io) as a professional profile with your resume, including your name, headshot, contact information, background, e.g., education, your experiences and skills (25 pts). - Create a link to a new HTML page to introduce this "Web Application Programming and Hacking" course and related hands-on projects (5 pts)

A website was created in a .html file utilizing a VM that contains the below requirements, formatted as to be both as functional and aethestically pleasing as possible given the current experience level. Included are the resume, name, headshot, contact, and background of the student. A new HTML page to introduce the course and related projects was also created.

###Non-technical requirements (20 pts) - Use an open-source CSS template or framework such as Bootstrap

Target this profile for your potential employer, and your page will be graded as a part of your job application

- Include a page tracker, for example: https://analytics.withgoogle.comLinks to an external site., https://flagcounter.com/Links to an external site..

A page tracker was also utilized via going to the link for flagcounter (above) and copy/pasting the code into the script of the aforementioned website.

Technical requirements (50 pts)

Basic JavaScript code (20 pts)

- Use jQuery and one more open-source JavaScript framework/library to implement JavaScript code introduced in Lab 2, including a digital clock, an analog clock, show/hide your email, and one more functionality of your choice. (5 pts each)
- Two public Web APIs integration (20 pts) Ensure you include a disclaimer stating that the public/third-party services generate the generated contents below and that you are not responsible.

1. Integrate the jokeAPI (https://v2.jokeapi.dev/joke/AnyLinks to an external site., similar to Lab 2.2.d.i) with `Any` category of joke to display a new joke in your page every 1 minute.
2. Integrate a public API with graphics and display that graphic/image in your page. Examples: https://xkcd.com/info.0.jsonLinks to an external site. (or https://xkcd.now.shLinks to an external site.), https://www.weatherbit.io, Links to an external site.https://dog.ceo/dog-apiLinks to an external site.

- Use JavaScript cookies to remember the client (10 pts): If first-time visit, display the message "Welcome to my homepage for the first time!";

otherwise, display the message "Welcome back! Your last visit was " (ensure that you update this value each time the same user visits -2pts if missing).

1. **jQuery and jQueryaJax were utilized to create, similar to Lab 2, a digital clock, an analog clock, to show/hide email upon clicking, and the bonus functionality (surprise!) in the website.**
2. **In addition, the jokeAPI from Lab 2 was also added, but its frequency was increased to every minute rather than every time the page reloads via making it a function that is called once every time the page is called, as well as every minute thereafter. Another API was also added, that is, Nasa's Astronomy Picture of the Day, that had to be formatted as either a photo or a link to a video. In future labs I hope to learn to optimize the loading of the website because it takes 20 seconds to load the image upon opening the website. A disclaimer was also made to shield the student of any blame from content added by these APIs.**
3. **Finally, JavaScript Cookies were utlized to tell when it is someone's first time visiting , or if they have come before, to welcome them back and display the last time they visited.**

Report

You must write a report using Markdown format following the template/outline provided in Lecture 2, which should include the course name and instructor, your name and email together with your headshot (150x150 pixels), and sub-sections of the assignment's overview, and each task and sub-task.

There should be an overview sub-section where you must write an overview of the assignment and the outcomes you learned from it. Include your website's clickable URL deployed on `github.io`. Also, include a direct clickable link to the project folder on GitHub.com so that it can be viewed when grading.

For each sub-task, write a brief summary of how you completed it. You are welcome to include code snippets and screenshots to demonstrate the outcome; however, they are not required. Please note that demo screenshots must include your name with proper captions and be visible, i.e., in high resolution, not too blurry, or with much blank space, for grading.

Your report must be exported in PDF with its contents and screenshots correctly rendered in proper order. The PDF file should be named `your-username-waph-project1.pdf`, e.g., `Hanchijd-waph-project1.pdf`, and uploaded to Canvas to be submitted by the deadline.

This report was written.

Deliverables and Submission

You need to submit three deliverables in PDF files for grading:

- Your report mentioned above.
- Your deployed website printed from a browser in PDF.
- The source code of your deployed website printed from a browser in PDF.

`pandoc` **was utilized to create the submitted documents that were thusly submitted to Canvas.**